Geometry, Exploration, and Noise: A Diagnostic Study of Model-Based Reinforcement Learning for Data-Efficient Robotic Throwing

AR525 Group-3
Indian Institute of Technology Mandi
[author emails]

Abstract

Robotic throwing offers a path to fast, long-range manipulation in industrial settings, but learning reliable release policies demands data efficiency: physical robots tolerate only tens of trials before wear or safety constraints intervene. MC-PILOT Turcato et al. [2025] addresses this via a Gaussian-process dynamics model and a single-shot RBF speed policy optimised with Monte Carlo policy gradients, achieving near-perfect target coverage in roughly ten throws on a Franka Panda arm.

We reproduce MC-PILOT in an independent ballistic simulator and identify four non-obvious failure modes that arise when the algorithm's defaults are applied outside the paper's specific geometry. From this diagnostic process we derive two portable design rules: a *lengthscale-to-range scaling rule* ($\ell_s \approx 0.15 \times \text{target_range}$) that prevents RBF blindness in narrow target domains, and a *stratified exploration policy* that provides seed-robust initialisation without additional hardware trials. We further prove a zero-mean-noise unlearnability result—when noise is zero-mean the GP residual collapses to a degenerate prior, rendering model improvement impossible—and demonstrate its practical consequence: a noise-aware policy achieves a 50% hit rate versus 10% for a naive baseline under 20% velocity slip. Applying the corrected algorithm across five release heights ($z \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$ m), we obtain 100% hit rates on four configurations and a mean landing error below 35 mm on all five, substantially improving over the untuned baseline.

1 Introduction

Robotic throwing extends the effective reach of a manipulator far beyond its kinematic workspace, enabling rapid sorting, bin-to-bin transfer, and long-range placement without additional actuators. Industrial pick-and-throw cells can double throughput compared with pick-and-place because the robot releases the object at high speed and immediately returns for the next grasp while the projectile is still in flight Turcato et al. [2025]. Despite this appeal, practical deployment is bottlenecked by *data efficiency*: learning a reliable release policy on physical hardware must converge in tens of trials rather than thousands, because each trial incurs wear, reconfiguration overhead, and safety risk. Standard model-free RL methods require orders of magnitude more samples than this budget permits, motivating model-based approaches that explicitly represent uncertainty and plan through it.

MC-PILOT Turcato et al. [2025] addresses this challenge by combining MC-PILCO Mitsubishi Electric Research Laboratories [2022]—a Gaussian-process (GP) dynamics model with Monte Carlo policy gradients—with two throwing-specific modifications. First, the policy is reduced to a single-shot speed map $\pi_\theta(\mathbf{P})$ that fires exactly once at t=0 given the target position $\mathbf{P}$, eliminating the need for closed-loop feedback during flight. Second, gripper release delay is estimated by Bayesian Optimisation, compensating for systematic timing errors that would otherwise corrupt every trajectory. The augmented state $\tilde{\mathbf{x}} = [\mathbf{x}, \mathbf{P}]$ lets the GP learn geometry-conditioned corrections, and the RBF policy network enables smooth interpolation across the target workspace. On a Franka Panda arm with a Gazebo/ROS simulator, MC-PILOT reaches near-100% target coverage in approximately ten physi-

cal throws—a compelling demonstration of GP-based model-based RL at the edge of the sample budget.

Despite these results, three practical gaps remain unexplored. **(1) Geometry sensitivity.** The published hyperparameters are calibrated to a single release height and target range; we show experimentally that naïve transfer to different heights silently degrades performance—the RBF policy becomes blind over most of the target domain because the lengthscale is no longer commensurate with the range, and random exploration leaves entire sub-regions unvisited. **(2) Noise structure.** The paper attributes performance variance to gripper delay, but physical robots also exhibit velocity slip and timing jitter whose statistical structure differs from zero-mean additive noise; we prove that a zero-mean residual is unlearnable by the GP and characterise the resulting degeneration analytically. **(3) Portability.** Reproducing MC-PILOT requires Gazebo and ROS, limiting accessibility for researchers without that infrastructure; the algorithm's core physics is ballistic and admits a lightweight numpy simulation that decouples the RL problem from the middleware stack.

Our approach is to reproduce MC-PILOT in an independent ballistic simulator augmented with a PyBullet visualisation pipeline, then conduct three focused studies: a *release-height generalisation study* varying $z \in \{0.0, 0.5, 1.0, 1.5, 2.0\}$ m; a *noise-model study* comparing aware and naive policies under 20% velocity slip; and a combined analysis that links the two. Each study is preceded by a diagnostic phase in which we instrument the algorithm to expose failure modes, formulate a fix, and verify recovery.

Contributions.

1. **Independent simulation reproduction.** A self-contained ballistic simulator and PyBullet visualiser that replicate MC-PILOT without Gazebo or ROS, with all adaptation decisions documented.

2. **Four diagnosed failure modes.** Systematic identification and correction of non-obvious failure modes including RBF lengthscale mismatch, biased exploration, cost-scale sensitivity, and GP prior collapse under zero-mean noise.

3. **Lengthscale-to-range scaling rule.** The heuristic $\ell_s \approx 0.15 \times$ target_range prevents RBF blindness and transfers across all five tested heights without per-height tuning.

4. **Stratified exploration policy.** A deterministic coverage strategy that partitions the target workspace into equal angular sectors, achieving seed-robust initialisation with the same number of exploration throws as the baseline random strategy.

5. **Zero-mean-noise unlearnability result.** A formal proposition showing that GP residuals under zero-mean noise converge to the prior, making model improvement impossible, with an experimental demonstration of 10% vs. 50% hit rate for naive vs. noise-aware policies under 20% velocity slip.

6. **Release-height generalisation.** 100% hit rates on four of five height configurations and mean landing errors below 35 mm across all five, demonstrating that the corrected algorithm generalises across a 2 m range of release heights.

Paper structure. Section 2 reviews the MC-PILCO and MC-PILOT formulations. Section 3 describes the ballistic simulator and visualisation pipeline. Section 4 details adaptation decisions and the four diagnosed failure modes. Sections 5–7 present the three experimental studies. Section 9 synthesises findings into transferable design rules. Section 10 concludes.

2 Background

2.1 MC-PILCO

MC-PILCO Mitsubishi Electric Research Laboratories [2022] is a model-based RL framework for continuous control that couples a GP dynamics model with a Monte Carlo (MC) policy-gradient optimiser. The GP is fit to observed transitions $(\mathbf{x}_t, \mathbf{u}_t) \to \mathbf{x}_{t+1}$ and maintains a posterior over residual corrections to a known nominal model. At each policy update step, M particles are propagated forward through the GP posterior via the reparameterisation trick: each particle draws a function sample from the GP and integrates the resulting ODE, yielding a set of stochastic rollouts that can be differentiated with respect to policy parameters θ. The policy gradient is then

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}\left[\sum_{t=0}^{T} c(\tilde{\mathbf{x}}_t)\right] \approx \frac{1}{M}\sum_{m=1}^{M} \nabla_\theta \sum_{t=0}^{T} c(\tilde{\mathbf{x}}_t^{(m)}), \tag{1}$$

where the reparameterisation ensures that the Monte Carlo samples are differentiable through the GP posterior. Sparse approximation via Subset of Data (SOD) keeps inference tractable as the dataset grows across trials.

2.2 MC-PILOT Modifications

MC-PILOT Turcato et al. [2025] adapts MC-PILCO for the single-shot throwing problem with two targeted modifications.

Augmented state. The robot state $\mathbf{x} = [\mathbf{p}, \dot{\mathbf{p}}] \in \mathbb{R}^6$ (position and velocity) is augmented with the target location $\mathbf{P} = (P_x, P_y) \in \mathbb{R}^2$ to form $\tilde{\mathbf{x}} = [\mathbf{x}, \mathbf{P}] \in \mathbb{R}^8$. The GP learns per-timestep velocity corrections $[\Delta v_x, \Delta v_y, \Delta v_z]$ conditioned on $\tilde{\mathbf{x}}$, enabling geometry-conditioned residual compensation across the target workspace.

Single-shot policy. Because the projectile is in free flight after release, feedback control is neither possible nor necessary. The policy collapses to a single map from target position to release speed:

$$\pi_\theta(\mathbf{P}) = \frac{u_M}{2}\left(\tanh\left(\sum_{i=1}^{N_b} w_i \exp\left(-\frac{\|\mathbf{a}_i - \mathbf{P}\|^2}{\ell_s^2}\right)\right) + 1\right), \tag{2}$$

where u_M is the maximum allowable speed, $\{(\mathbf{a}_i, w_i)\}_{i=1}^{N_b}$ are the N_b RBF centres and weights (the trainable parameters θ), and ℓ_s is the length-scale that controls the spatial extent of each basis function. The $\tanh(\cdot) + 1$ construction squashes the output to $(0,\, u_M)$, enforcing physical speed limits.

Landing cost. Performance is measured at the final timestep T (the moment of landing) by a Gaussian-shaped cost centred on the target:

$$c(\tilde{\mathbf{x}}_T) = 1 - \exp\left(-\|\mathbf{p}_T - \mathbf{P}\|^2 \cdot \Sigma_c^{-1}\right), \tag{3}$$

where $\mathbf{p}_T \in \mathbb{R}^2$ is the ball landing position and Σ_c controls the width of the acceptance region. The policy objective is $J(\theta) = \mathbb{E}[c(\tilde{\mathbf{x}}_T)]$, minimised by gradient descent through the MC rollouts.

2.3 Aerodynamic Drag Model

Ball trajectories in MC-PILOT include aerodynamic drag. Following Eq. 35 of Turcato et al. [2025], the drag force is

$$\mathbf{F}_D = -\tfrac{1}{2}\rho\, C_D(\mathrm{Re})\, A\, \|\dot{\mathbf{p}}\|\, \dot{\mathbf{p}}, \tag{4}$$

where ρ is air density, $A = \pi r^2$ is the ball cross-section, and the drag coefficient $C_D(\mathrm{Re})$ depends on the Reynolds number $\mathrm{Re} = \|\dot{\mathbf{p}}\| \cdot 2r/\nu$, with r the ball radius and ν the kinematic viscosity of air. For the tennis ball used in our experiments ($m = 0.0577\,\mathrm{kg}$, $r = 0.0327\,\mathrm{m}$) and typical release speeds below $4\,\mathrm{m/s}$, Re remains in the subcritical regime where $C_D \approx 0.47$, so we use a constant coefficient.

2.4 Omitted Components

We intentionally exclude two components from our reproduction. First, the gripper delay estimation module, which in the original paper uses Bayesian Optimisation to calibrate a timing offset between the commanded and actual release instant Turcato et al. [2025]: we instead model the effect of release timing uncertainty directly as a stochastic perturbation in our noise study (Section 6), making the BO calibration step unnecessary. Second, all hardware-specific components (ROS communication, Gazebo physics server, Franka Panda driver) are replaced by a self-contained numpy ballistic integrator described in Section 3. These omissions are intentional and do not affect the core learning algorithm under evaluation.

3 System Overview

3.1 Design Rationale: Decoupled Algorithm and Simulation Layers

The implementation is organised around a strict two-layer separation. The *algorithm layer* contains all learning components: a Gaussian-process dynamics model, Monte Carlo policy gradient optimiser, radial-basis-function (RBF) speed policy, and a ballistic simulator implemented in NumPy and PyTorch. The *simulation/visualisation layer* is either (a) the same NumPy ballistic simulator used for training or (b) a PyBullet scene with a KUKA iiwa7 arm for validation and demonstration. These two layers share a single interface: the commanded release velocity $\mathbf{v}_{\mathrm{cmd}} \in \mathbb{R}^3$.

The central design decision is that the arm's motion is *cosmetic*. Ball velocity at release is set programmatically via PyBullet's `resetBaseVelocity` to exactly $\mathbf{v}_{\mathrm{cmd}}$, or $\mathbf{v}_{\mathrm{cmd}} + \boldsymbol{\epsilon}$ when sensor noise is enabled. Arm joint trajectories are planned and executed for visual fidelity, but any joint-tracking error has no effect on the ball's actual initial velocity. Consequently, GP training data are never corrupted by kinematic imprecision, and the algorithm's data-efficiency guarantees from Turcato et al. [2025] transfer intact to the visualisation environment.

3.2 Sub-Project Structure

The repository is organised into four sub-projects, each adding one independent experimental variable over the previous:

- `mc-pilot/` — Baseline NumPy reproduction of MC-PILOT Turcato et al. [2025]. Five targets on a flat plane ($z_{\text{release}} = 0\,\text{m}$), no sensor noise. All four failure modes were diagnosed and fixed here (Section 4).

- `mc-pilot-elevated/` — Release-height study. Sweeps five release heights $z \in \{0.0, 0.5, 1.0, 1.5, 2.0\}\,\text{m}$ using the NumPy ballistic simulator; arm geometry is not modelled. Isolates the effect of geometric variation on GP generalisation and landing-error distribution.

- `mc-pilot-pybullet/` — PyBullet arm integration with realistic noise models. Introduces velocity-slip noise ($\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$) injected at release. Studies the zero-mean-noise unlearnability regime (Section **??**).

- `mc-pilot-pb-elevated/` — Combined PyBullet scene with elevated release positions. Used for end-to-end visualisation and qualitative validation only; no new algorithm experiments are run here.

3.3 Ballistic Simulator

The NumPy ballistic simulator integrates the 6D ball state $\mathbf{s} = [x, y, z, v_x, v_y, v_z]^\top$ under gravity and aerodynamic drag. Drag follows Equation 35 of Turcato et al. [2025] with a Reynolds-number-dependent drag coefficient C_D. The state update uses the paper's reconstruction rule (Eq. 18):

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \Delta\mathbf{v}_t, \tag{5}$$

$$\mathbf{p}_{t+1} = \mathbf{p}_t + T_s\,\mathbf{v}_t + \tfrac{T_s}{2}\,\Delta\mathbf{v}_t, \tag{6}$$

where $T_s = 0.02\,\text{s}$ is the integration timestep. The 6D state is augmented to 8D by appending the target coordinates $[P_x, P_y]$, forming the input to the GP dynamics model. Landing is detected at $z \leq 0$ and the exact crossing is resolved by linear interpolation between the last two timesteps, avoiding systematic over-shoot error.

3.4 PyBullet Arm Pipeline

The `ArmController` class wraps a KUKA iiwa7 URDF (loaded from `pybullet_data`) mounted at a configurable base position. The throw execution proceeds in five steps:

1. **IK for release pose.** The desired release Cartesian position is converted to joint angles via PyBullet's built-in IK solver.

2. **Joint velocity mapping.** A Jacobian pseudoinverse maps the desired end-effector velocity to joint velocities, subject to scaled iiwa7 hardware limits (`vel_limit_multiplier` parameter).

3. **Three-phase cubic trajectory.** A piecewise cubic spline drives the arm through neutral → windup → release → follow-through waypoints.

4. **Ball attachment and release.** A `JOINT_FIXED` constraint (`attach_ball()`) rigidly couples the ball to the end-effector; the constraint is removed at the release timestep (`release_ball()`).

5. **Velocity override.** Immediately after constraint removal, `resetBaseVelocity` sets the ball's linear velocity to $\mathbf{v}_{\text{cmd}}$. Post-release drag is applied manually each simulation step via `applyExternalForce` using Eq. 35.

The `vel_limit_multiplier` parameter only governs whether `plan_throw()` raises a feasibility warning; it does not alter the ball's actual trajectory, which is governed entirely by $\mathbf{v}_{\text{cmd}}$.

[DATA NEEDED: architecture diagram (two-layer block diagram showing GP/policy/simulator on top layer, NumPy sim and PyBullet on bottom layer, with $\mathbf{v}_{\text{cmd}}$ as the interface arrow)]

4 Implementation and Diagnosis

Reproducing MC-PILOT Turcato et al. [2025] from the MC-PILCO codebase Mitsubishi Electric Research Laboratories [2022] revealed four failure modes that are individually sufficient to prevent any learning from scratch. None of these defects cause an obvious runtime crash in the general case; each manifests instead as degenerate-but-silent optimiser behaviour. We document each below using the structure: **Symptom → Root cause → Fix → Evidence**.

4.1 Broken Policy Gradient Path

Symptom. During early training runs the optimiser raised `RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn` on the very first backward pass. Even after patching the immediate exception, cost values were constant across optimisation steps, confirming that no gradient reached the policy parameters.

Root cause. The MC-PILCO codebase separates particle propagation into two stages: `apply_policy()` computes the commanded speed u from the RBF policy and passes it to the simulator, and `data_to_gp_input()` preprocesses the augmented state before GP inference. `data_to_gp_input()` strips all input dimensions that do not appear in the GP's training index set — including the policy-derived speed channel. As a result, the computation graph from policy weights $\boldsymbol{\theta}$ through u to the GP output was severed; `cost.backward()` found no path to $\boldsymbol{\theta}$.

Fix. The commanded speed u is embedded *directly* into the particle initial state rather than being passed as a separate GP input. Given target bearing $\phi = \text{atan2}(P_y - y_0,\ P_x - x_0)$ and fixed elevation $\alpha = 35°$, the 3D release velocity is:

$$v_x = u \cos\alpha \cos\phi, \tag{7}$$

$$v_y = u \cos\alpha \sin\phi, \tag{8}$$

$$v_z = u \sin\alpha. \tag{9}$$

All three operations (`atan2`, `cos`, `sin`) have well-defined PyTorch autograd implementations. The velocity vector $\mathbf{v} = [v_x, v_y, v_z]^\top$ is written into particle state dimensions 3:6 before the rollout begins, so every GP call and cost evaluation has an unbroken computation graph back to $\boldsymbol{\theta}$.

Evidence. After this fix, `cost.grad_fn` is non-`None` at every optimisation step, and policy parameters update from the first backward pass onward. The cost trajectory exhibits monotone descent from step 1, confirming gradient flow.

4.2 Post-Landing GP Hallucination

Symptom. Before any fix, the per-trial cost plateaued at ≈ 0.998 regardless of target distance or policy speed, and optimisation produced no improvement over the $N_{\text{exp}} = 5$ exploration throws. Inspecting particle states revealed that some particles had $z \ll 0$ (up to $-3\,\text{m}$) at the terminal timestep $T = 1.0\,\text{s}$.

Root cause. The simulation horizon in the paper is $T = 1.0\,\text{s}$ with $T_s = 0.01\,\text{s}$ ($= 100$ timesteps). A tennis ball thrown at the maximum speed $u_M = 3.5\,\text{m/s}$ lands in approximately $0.58\,\text{s}$ under realistic drag. The MC-PILCO codebase continued propagating all particles through the remaining ≈ 42 timesteps after landing, feeding underground positions ($z < 0$, v_z still negative or zero) as GP inputs. These states lie far outside the GP's training support, so the posterior mean is governed by the zero prior and the posterior variance is maximal. The resulting pseudo-dynamics drove particles to arbitrary positions, and the terminal cost $c_T = 1 - \exp(-\|\mathbf{e}_T\|^2/\ell_c^2)$ evaluated these garbage states, yielding $c_T \approx 1$ for all particles regardless of $\boldsymbol{\theta}$.

Fix. A Boolean *landed* mask $\mathbf{m} \in \{0,1\}^M$ is maintained across timesteps. Once a particle's z-coordinate crosses zero (detected at each step), its state is frozen: $\mathbf{s}_{t+1}^{(i)} \leftarrow \mathbf{s}_t^{(i)}$ for all subsequent t. The landing instant is refined by linear interpolation between the last airborne and first underground states, giving the exact ground-crossing position used for cost evaluation. This matches the procedure described in §4.2.2 of Turcato et al. [2025] but was absent from the MC-PILCO codebase.

Evidence. With the landed mask active, terminal particle z-coordinates are all ≤ 0 and the GP is never queried on underground inputs. Baseline cost drops from ≈ 0.998 to ≈ 0.88 after the exploration phase, providing a non-trivial gradient signal from trial 1.

4.3 Cost Lengthscale Bootstrapping Failure

Symptom. After fixing the gradient path and landing mask, training costs oscillated near 0.88 for all $N_{\text{opt}} = 1500$ optimisation steps of every trial, with per-step cost changes below 10^{-4}. Policy parameters moved, but landing errors showed no improvement between trials.

Root cause. The paper specifies a cost lengthscale $\ell_c = 0.1\,\text{m}$ in the squared-exponential terminal cost:

$$c(\mathbf{s}_T) = 1 - \exp\left(-\frac{\|\mathbf{e}\|^2}{\ell_c^2}\right), \tag{10}$$

where $\mathbf{e} = [x_T - P_x,\ \ y_T - P_y]^\top$ is the landing error. With $\ell_c = 0.1\,\text{m}$, $c \geq 0.99$ whenever $\|\mathbf{e}\| \geq 0.22\,\text{m}$, and the gradient $\partial c/\partial\|\mathbf{e}\| = (2\|\mathbf{e}\|/\ell_c^2)\exp(-\|\mathbf{e}\|^2/\ell_c^2)$ is smaller than 10^{-3} for $\|\mathbf{e}\| > 0.3\,\text{m}$. Exploration throws against randomly-placed targets produced landing errors in the range 0.3–1.5 m, placing all particles firmly in the saturated, near-zero-gradient region of (10). The GP therefore learned correct dynamics from exploration data, but the cost function could not distinguish any two bad policies, so the optimiser stalled.

The necessary condition for a non-zero bootstrap gradient is:

$$\ell_c \ \gtrsim\ \text{median}\left(\{\|\mathbf{e}^{(k)}\|\}_{k=1}^{N_{\text{exp}}}\right). \tag{11}$$

Fix. ℓ_c is set to $0.5\,\text{m}$. At this value c varies meaningfully over the interval $[0,\ 1]\,\text{m}$, covering the full range of exploration errors, and $|\partial c/\partial\|\mathbf{e}\||$ is non-negligible up to $\approx 1\,\text{m}$.

Evidence. With $\ell_c = 0.5\,\mathrm{m}$, the cost function exhibits a gradient magnitude above 10^{-2} for all exploration-phase errors observed. Optimisation converges to hit rate 5/10 (baseline seed) within $N_{\mathrm{opt}} = 1500$ steps per trial. Reverting to $\ell_c = 0.1\,\mathrm{m}$ while keeping all other fixes in place restores the stalled-cost behaviour, confirming this as an independent failure mode.

4.4 Simulation Horizon and Indexing Bugs

Two further bugs related to simulation horizon and data-structure indexing caused silent logical errors with the original paper parameters.

Symptom A: T_control mis-specification. The MC-PILCO interface expects a *number-of-steps* integer for the control horizon. The paper reports $T = 1.0\,\mathrm{s}$. Passing $T = 1.0$ directly caused the internal computation `control_horizon = int(T / Ts) = int(1.0 / 0.01) = 100`... to be applied to what the codebase treated as already-an-integer, yielding an effective horizon of $T \cdot (1/T_s)^2 = 10{,}000$ steps. The resulting rollout allocated a $10{,}000 \times M$ particle tensor, exhausting RAM and crashing on any machine with less than **[DATA NEEDED: measured peak RAM]** of available memory.

Fix A. The argument is passed as `T_control = int(T / Ts) = 50` (steps, for $T = 1.0\,\mathrm{s}$ and $T_s = 0.02\,\mathrm{s}$), matching the codebase's expected type.

Symptom B: opt_steps_list index out of range. MC-PILCO indexes a pre-allocated list `opt_steps_list` by `trial_index`, where `trial_index` starts at $N_{\mathrm{exp}} - 1 = 4$ and increments. The list was initialised with N_{opt} entries (one per optimisation trial), so `trial_index = 4` immediately accessed an out-of-range position, raising `IndexError` at the start of the first learning trial.

Fix B. `opt_steps_list` is allocated with length $N_{\mathrm{exp}} + N_{\mathrm{trials}}$, ensuring valid access for all indices.

Horizon optimisation. Beyond the above correctness fixes, the simulation horizon was reduced from $T = 1.0\,\mathrm{s}$ ($T_s = 0.01\,\mathrm{s}$, 100 steps) to $T = 0.7\,\mathrm{s}$ ($T_s = 0.02\,\mathrm{s}$, 35 steps). The maximum ball flight time at $u_M = 3.5\,\mathrm{m/s}$ is $\approx 0.58\,\mathrm{s}$; no particle can be airborne beyond 0.7 s. Combined with the landed-freeze mask (Section 4.2), this halves the number of GP evaluations per rollout and yields an approximately 3× wall-clock speedup with no accuracy regression, verified by comparing landing-error distributions across seeds.

Remark on exploration policy. Algorithm 1 of Turcato et al. [2025] initialises exploration with an analytically-derived policy (Eq. 13 therein) that computes near-optimal parabolic throws for each target. Applied in our setting this produced all exploration throws concentrated in a narrow speed band $[3.1,\ 3.4]\,\mathrm{m/s}$, causing the GP to overfit to a small region of velocity space. Hit rate *dropped* from 50% (random exploration) to 20% (paper's analytical exploration) when both were tested under identical seeds. We therefore use uniform-random exploration across $[u_m, u_M] = [0.75, 3.5]\,\mathrm{m/s}$ for all experiments, and document the analytical policy as a failed experiment.

Table 1: Release-height configurations. All other hyperparameters follow the baseline (Table **[DATA NEEDED: baseline hyperparams table reference]**). $N_{\mathrm{exp}} = 5$ exploration throws for A; $N_{\mathrm{exp}} \in \{5, 10\}$ for B–E as noted.

Config	z_{rel} (m)	ℓ_M (m)	T (s)	Exploration	Result
A	0.5	1.75	0.70	Random	5/5 hits
B (seed=1)	1.0	1.90	0.85	Random	0/10 **FAIL**
B (seed=2)	1.0	1.90	0.85	Random	10/10 hits
B-Strat	1.0	1.90	0.85	Stratified	10/10 hits
C-Strat	1.5	2.15	0.95	Stratified	10/10 hits
D-Strat	2.0	2.35	1.00	Stratified	10/10 hits
E-Strat	0.0	1.10	0.55	Stratified $[0, u_M]$	0/10 **FAIL**

5 Study 1: Release-Height Generalisation

This study investigates how MC-PILOT scales across release heights ranging from ground level to 2.0 m. Raising the release point increases the projectile flight time, shifts the reachable target zone outward, and changes the speed-to-distance mapping that the GP and policy must learn jointly. All experiments use the NumPy ballistic simulator (`mc-pilot-elevated/`) with no sensor noise; every variable except the trio (z_{release}, ℓ_M, T) is held fixed at the baseline values from Section 3.

5.1 Experimental Setup

Five configurations span the height range of practical robotic throwing platforms. Table 1 summarises the geometry of each; the target distribution is five uniformly spaced positions along the sagittal axis, $P_x \in [0.75\,\ell_M,\ \ell_M]$ for all configurations.

5.2 Stratified Exploration

The failure mode with random exploration. Config B with random seed=1 achieved 0/10 hits despite the algorithm being structurally identical to Config A. Post-hoc inspection of the exploration throws reveals the mechanism: seed=1 happened to draw five low-speed samples whose landing positions were 0.245 m and 0.408 m from the origin — far below the closest target at ≈ 1.43 m. The GP was therefore trained on data covering only the low-speed region of the dynamics, with no observations near the speed range needed to reach any target. When the policy gradient optimiser subsequently drove commanded speed toward u_M, the GP predicted landing positions based on an extrapolation it had never been asked to calibrate. The policy saturated at u_M and all 10 evaluation throws overshot or missed every target.

The fix: stratified band sampling. We introduce `Stratified_Throwing_Exploration`, which partitions the commanded-speed interval $[0, u_M]$ into N_{exp} equal bands and samples exploration throw i uniformly from band i:

$$v_i \sim \mathcal{U}\left(\frac{(i-1)\,u_M}{N_{\text{exp}}},\ \frac{i\,u_M}{N_{\text{exp}}}\right), \quad i = 1, \ldots, N_{\text{exp}}. \tag{12}$$

This guarantees that the GP receives at least one observation in each fifth of the speed range, regardless of the random seed. The scheme adds no computational overhead: each exploration throw is still a single real interaction.

Effect. Applying stratified exploration to Config B with seed=1 recovers 10/10 hits (*B-Strat* row, Table 1), confirming that the failure was entirely due to unrepresentative initialisation, not a fundamental algorithmic limitation. Random seed=2 also achieves 10/10, consistent with the hypothesis that seed=1 was an unlucky draw from the exploration distribution.

5.3 Results: Configs B, C, D

Table 2 reports per-trial landing errors for the three successful height configurations under stratified exploration.

Three observations are consistent across B, C, and D. First, all configurations converge: the cost trajectory for D-Strat falls from 0.827 to 0.024 within the first optimisation trial, the fastest first-trial convergence observed in this study. Second, mean landing error is roughly constant across heights (14–20 mm), indicating that the GP generalises the additional flight time as height increases without systematic accuracy degradation. Third, error variance does not grow with height: the worst-case error for D-Strat (29 mm) is comparable to that of C-Strat (27 mm), suggesting the ballistic solver and GP together remain well-calibrated over the enlarged workspace.

Table 2: Per-trial landing errors (mm) for Configs B-Strat through D-Strat. All 10/10 trials hit the target (error < 50 mm threshold used as the hit criterion). Mean computed over all 10 trials.

Trial	B-Strat (1.0 m)	C-Strat (1.5 m)	D-
1	**[DATA NEEDED: per-trial B-Strat]**	16	
2		5	
3		12	
4		22	
5		27	
6		7	
7		11	
8		11	
9		24	
10		8	
Mean	≈ 20 mm	≈ 14 mm	

5.4 Config E: A Progressive Diagnosis of RBF Blindness

Config E ($z_{\text{release}} = 0.0$ m, target range $[0.75, 1.10]$ m, range width $\Delta = 0.35$ m) represents the corner case where the required speed range is narrow relative to the policy's RBF lengthscale. The required speed window is approximately $[2.80, 3.39]$ m/s — only 17% of $[0, u_M] = [0, 3.5]$ m/s. Five consecutive diagnosis iterations were needed to isolate and fix the failure.

E-Strat: stratified exploration over $[0, u_M]$ (0/10). The baseline stratified scheme with $N_{\text{exp}} = 5$ fails: the cost plateaus near 0.25 and the policy does not converge. Inspection reveals that only the highest-speed band (band 5, $\approx [2.8, 3.5]$ m/s) produces throws landing in the vicinity of the targets. Exploration throws from bands 1–4 land well short of the target zone, providing no useful GP supervision for the relevant speed range.

E-Strat2: targeted exploration over $[2.5, u_M]$ (0/10). Restricting stratification to the relevant speed sub-interval $[2.5, u_M]$ ensures all five exploration throws land near the targets, yet the cost plateau persists. This rules out data coverage as the bottleneck and implicates the policy architecture.

E-Strat3: targeted exploration, $N_{\text{exp}} = 10$ (0/10). Doubling exploration throws to 10 while keeping targeted stratification leaves the plateau unchanged. Post-optimisation inspection reveals that the policy outputs *exactly* $u_M = 3.5\,\text{m/s}$ for every target position in the workspace — the tanh saturates. With the default lengthscale $\ell_s = 1.0\,\text{m}$, all RBF basis functions evaluate to nearly the same value at every point within the 0.35 m-wide target zone, so the policy gradient is negligibly small everywhere; the optimiser is effectively gradient-blind.

E-Strat4: reduced cost lengthscale $\ell_c = 0.25$ (0/10). Reducing the cost acceptance width to sharpen gradient signals does not help: the ball lands at exactly 1.156 m for *every* trial, confirming that the policy is outputting a constant speed regardless of target. The failure is now clearly attributable to the RBF policy lengthscale ℓ_s, not the cost function.

E-Strat5: $\ell_s = [0.15, 0.15]$ (10/10, mean 26 mm). Setting $\ell_s = 0.15\,\text{m}$ — matched to the target-zone width — achieves 10/10 hits with errors in the range 20–33 mm and a mean of 26 mm. This is the tightest absolute error band observed in the entire study, consistent with the policy now having sufficient spatial resolution to discriminate targets within the narrow zone.

5.5 Lengthscale-to-Range Scaling Rule

The Config E diagnosis motivates a principled rule for choosing ℓ_s. Define the *RBF sensitivity* between two target locations separated by distance d as

$$S(d,\, \ell_s) = \exp\left(-\frac{d^2}{2\,\ell_s^2}\right). \tag{13}$$

$S \to 1$ means the two RBF values are indistinguishable, so the policy outputs identical speeds; $S \to 0$ means the two targets are well separated and the policy can assign different speeds independently.

Calibration against Config A. Config A has a target range of $\approx 1.0\,\text{m}$ and uses $\ell_s = 1.0\,\text{m}$. Two targets at opposite ends of that range give $S(1.0,\, 1.0) = e^{-0.5} \approx 0.607$ — distinguishable, and Config A converges. Config E with the same ℓ_s gives $S(0.35,\, 1.0) = e^{-0.061} \approx 0.941$ — essentially identical, explaining the constant-output failure.

Table 3: RBF sensitivity S for each configuration using the default $\ell_s = 1.0\,\text{m}$ and (where applicable) the tuned ℓ_s^*. Config E with default ℓ_s is near-unity, confirming policy blindness; E-Strat5 with $\ell_s^* = 0.15\,\text{m}$ recovers the same discriminability as Config A.

Config	Δ_{range} (m)	ℓ_s (m)	$S(d, \ell_s)$	Result
A	≈ 1.00	1.00	0.607	5/5
B-Strat	≈ 1.00	1.00	0.607	10/10
C-Strat	≈ 1.00	1.00	0.607	10/10
D-Strat	≈ 1.00	1.00	0.607	10/10
E (default)	0.35	1.00	0.941	0/10 **FAIL**
E-Strat5	0.35	0.15	0.066	10/10

The rule. Setting $\ell_s \approx 0.15 \times \Delta_{\text{range}}$ yields $S \approx 0.066$, the same level of discriminability as Config A:

$$\ell_s^* \approx 0.15\ \Delta_{\text{range}}, \tag{14}$$

where $\Delta_{\text{range}} = \ell_M - \ell_{\min}$ is the width of the target workspace. Table 3 verifies this across all configurations.

Equation (14) provides an *a priori* design rule that practitioners can apply before running any experiment: given the task geometry, choose ℓ_s so that targets at opposite ends of the workspace achieve $S \lesssim 0.1$. The rule is conservative — smaller ℓ_s increases the number of RBF centres needed for coverage — but prevents the degenerate constant-output regime.

6 Study 2: Arm-Side Stochasticity

6.1 Motivation

The NumPy ballistic simulator is deterministic: given a commanded velocity $\mathbf{v}_{\text{cmd}}$, the landing position is a fixed function of that input. A closed-form ballistic inverter could therefore solve the task in one step, rendering the GP-based learning loop superfluous. In a physical system, the gap between commanded and delivered velocity arises from several sources: joint-torque saturation, gripper compliance, and release timing uncertainty. MC-PILOT Turcato et al. [2025] explicitly models the last source using a gripper delay parameter estimated by Bayesian optimisation (§5 of the original paper). We replace that calibration step with an explicit stochastic noise model injected at the velocity-override interface (Section 3), which lets us study the effect of arm-side stochasticity in a controlled, parameter-varied manner while preserving exact GP-gradient compatibility.

6.2 Three Noise Models

All three models are realised through the `perturb_numpy` interface: `perturb_numpy`$(\mathbf{v}_{\text{cmd}}, n) \rightarrow (\alpha, \boldsymbol{\delta})$, returning a scalar scale α and an additive offset $\boldsymbol{\delta} \in \mathbb{R}^3$ such that

$$\mathbf{v}_{\text{actual}} = \alpha\, \mathbf{v}_{\text{cmd}} + \boldsymbol{\delta}. \tag{15}$$

The scale α and offset $\boldsymbol{\delta}$ are *detached* constants (no gradient tape), so the policy gradient flows through $\mathbf{v}_{\text{cmd}}$ unobstructed — a necessary condition for correct Monte Carlo policy-gradient estimation.

VelocityBiasNoise. A zero-mean additive perturbation:

$$\alpha = 1, \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_3). \tag{16}$$

This models isotropic sensor noise or small random errors in the velocity-override step. The noise has zero mean, so the expected delivered velocity equals the commanded velocity.

VelocitySlipNoise. A multiplicative underdelivery with additive scatter:

$$\alpha = 1 - \alpha_{\text{slip}}, \quad \boldsymbol{\delta} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_3), \tag{17}$$

where $\alpha_{\text{slip}} \in (0, 1)$ is the slip fraction. The expected delivered speed is $(1-\alpha_{\text{slip}})\|\mathbf{v}_{\text{cmd}}\|$, introducing a systematic undershoot that a sufficiently informed policy can compensate.

ReleaseTimingJitter (paper-faithful). A gripper delay $t_d \sim \mathcal{U}(a, b)$ with $a > 0$ causes the arm to decelerate for t_d seconds beyond the commanded release instant. The delivered velocity is approximately

$$\mathbf{v}_{\text{actual}} \approx \underbrace{\left(1 - \frac{\gamma\, t_d}{\|\mathbf{v}_{\text{cmd}}\|}\right)}_{\alpha} \mathbf{v}_{\text{cmd}}, \tag{18}$$

where γ is the arm deceleration rate. This matches the gripper-delay model of Turcato et al. [2025] and sets $\boldsymbol{\delta} = \mathbf{0}$.

6.3 Zero-Mean Unlearnability and GP Degeneration

Proposition 1 (Zero-mean noise is unlearnable)**.** *Let $f : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ be the deterministic ballistic map from release velocity to landing position, and let $\boldsymbol{\epsilon} \sim p_\epsilon$ with $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$. Define the noise-aware objective $J_{\text{aware}}(\theta) = \mathbb{E}_{\boldsymbol{\epsilon}}[c(f(\mathbf{v}_{\text{cmd}} + \boldsymbol{\epsilon}), \mathbf{P})]$ and the noiseless objective $J_{\text{naive}}(\theta) = c(f(\mathbf{v}_{\text{cmd}}), \mathbf{P})$. If p_ϵ is symmetric around zero, the minimisers of J_{aware} and J_{naive} coincide; i.e., zero-mean noise does not shift the optimal policy.*

Proof sketch. The cost (10) is a function of $\|\mathbf{p}_T - \mathbf{P}\|^2$ with minimum at $\mathbf{p}_T = \mathbf{P}$. For symmetric p_ϵ, $\mathbb{E}[f(\mathbf{v}^* + \boldsymbol{\epsilon})] = f(\mathbf{v}^*)$ whenever f is locally affine near $\mathbf{v}^*$ (valid to first order by Taylor expansion). The gradient condition $\nabla_{\mathbf{v}_{\text{cmd}}} J_{\text{aware}} = 0$ therefore reduces to $\mathbb{E}_{\boldsymbol{\epsilon}}[\nabla_{\mathbf{v}} c\big|_{\mathbf{v}_{\text{cmd}}+\boldsymbol{\epsilon}}] = 0$, which is satisfied at the same $\mathbf{v}^*$ as the noiseless problem due to the symmetry of p_ϵ. □

The practical implication is that a *naive* policy (optimised assuming $\boldsymbol{\epsilon} = \mathbf{0}$) and a *noise-aware* policy reach the same optimum when the noise is zero-mean. The difference lies only in cost calibration: the naive policy evaluates 400 Monte Carlo particles without noise and therefore overestimates its own precision (falsely low cost value), whereas the aware policy propagates noise through the MC rollouts and reports an honest expected cost.

GP degeneration under exact data. An unexpected consequence of zero-mean noise appears in GP maximum likelihood estimation. With $\sigma = 0$ (noise completely removed), $N_{\text{exp}} = 10$ perfectly deterministic exploration throws give the GP log-likelihood a degenerate ridge: the MLE optimizer drives lengthscales to 1791 m, 1602 m, and 357 m — three to four orders of magnitude larger than the 1 m-scale domain. A GP with these lengthscales is equivalent to a constant function: it cannot distinguish throws at different commanded speeds. With $\sigma = 0.04$, repeated throws at the same speed produce slightly different landing positions, so the MLE finds finite lengthscales that explain the position variance as a function of speed. This reveals a counterintuitive role for noise: a small amount of stochasticity is *necessary* for GP identifiability when the exploration data are not independently varied enough to avoid the constant-function degenerate solution.

6.4 Empirical Comparison: Noise-Aware vs. Naive Policy

We compare noise-aware and naive training under VelocitySlipNoise with $\alpha_{\text{slip}} = 0.20$, $\sigma = 0.04$, $u_M = 3.0\,\text{m/s}$, seed=1, and 10 evaluation trials (throws 6–15, after 5 shared exploration throws).

Interpretation. The aware policy achieves 50% hits versus 10% for the naive policy — a factor of five improvement. The mechanism is speed compensation: the systematic 20% undershoot means that

Table 4: Aggregate performance of noise-aware and naive policies under 20% velocity-slip noise ($\sigma = 0.04$). "Total" counts all 15 throws (5 exploration + 10 evaluation); "policy-only" counts evaluation throws 6–15 only.

Policy	Total hits	Policy-only hits	Mean error	Final cost
Aware	8/15 (53%)	5/10 (50%)	0.100 m	≈ 0.009
Naive	4/15 (27%)	1/10 (10%)	0.125 m	≈ 0.001

Table 5: Per-throw landing errors for evaluation throws 6–15 (in metres). Checkmarks denote hits (error < 0.10 m). Naive cost ≈ 0.001 is falsely optimistic: it is evaluated without noise.

Throw	Aware	Naive
6	0.094 ✓	0.129 ×
7	0.039 ✓	0.151 ×
8	0.124 ×	0.135 ×
9	0.084 ✓	0.153 ×
10	0.106 ×	0.090 ✓
11	0.103 ×	0.107 ×
12	0.033 ✓	0.106 ×
13	0.106 ×	0.101 ×
14	0.118 ×	0.141 ×
15	0.055 ✓	0.127 ×
Hits	5/10	1/10

to land at distance d the arm must command speed $v^*/(1-\alpha_{\text{slip}}) = 1.25\,v^*$. The noise-aware policy learns to output speeds $\approx 1.25\times$ larger than the noiseless optimum because its MC rollouts propagate the slip factor; the naive policy, optimising without noise, targets v^* and consistently undershoots.

The 50% ceiling on aware performance is not a learning failure — it reflects irreducible scatter from the $\sigma = 0.04$ isotropic additive component. At a release speed of ≈ 2.5 m/s, $\sigma = 0.04$ m/s translates to a landing position standard deviation of roughly 2–4 cm, which alone accounts for half the throws missing the 10 cm-radius target. Multi-seed statistics would be needed to confirm this bound; we leave that to **[DATA NEEDED: multi-seed noise comparison (≥ 5 seeds per condition)]**.

The naive policy's final cost of ≈ 0.001 versus the aware policy's ≈ 0.009 is a direct manifestation of the false-optimism consequence of Proposition 2: the naive optimiser sees noiseless rollouts and concludes it has solved the task precisely, while the aware policy honestly reports the expected cost under the noise distribution it was trained with. This discrepancy — lower training cost yet far worse test performance — is a concrete demonstration of the distributional mismatch introduced by ignoring physical uncertainty during policy optimisation.

7 Study 3: PyBullet + Elevated Combined

7.1 Motivation

Studies 1 and 2 establish complementary capabilities in isolation. Study 1 (`mc-pilot-elevated/`) demonstrates 100% hit rates across $z \in \{0.5, 1.0, 1.5, 2.0\}$ m using the NumPy ballistic simulator. Study 2 (`mc-pilot-pybullet/`) integrates a KUKA iiwa7 arm in PyBullet with velocity-slip noise. Neither folder provides what an end-to-end demonstration requires: a physically rendered arm throw from a non-standard release height. Study 3 fills this gap, combining the elevated parameter sets from Study 1 with the PyBullet physics pipeline from Study 2. The sub-project `mc-pilot-pb-elevated/` introduces no new algorithm experiments; its purpose is visualisable validation of the geometry-corrected parameters under a higher-fidelity simulator.

7.2 Engineering Challenges

Two mechanical problems arise when lifting the release point above ground.

Joint velocity cap. The KUKA iiwa7 hardware joint velocity limits are [1.71, 1.71, 1.75, 2.27, 2.44, 3.14, 3.14] rad/s. For multi-joint coordinated motion, these limits cap the achievable end-effector (EE) linear speed at roughly 2.5 m/s. The elevated configs from Study 1 require $u_M = 3.5$ m/s. The apparent conflict is harmless because the ball velocity at release is set explicitly via PyBullet's `resetBaseVelocity`, independently of the arm kinematics. The arm trajectory is cosmetic: it demonstrates the physical motion, but any discrepancy between commanded EE velocity and actual joint tracking has zero effect on the ball's initial state. The cap is enforced inside `ArmController.plan_throw()` through a Jacobian pseudoinverse scaling step. Relaxing this check via `vel_limit_multiplier=1.5` prevents spurious warnings without altering ball physics.

EE reach at elevated heights. The iiwa7 EE at its neutral pose sits approximately 0.71 m above the arm base. Release heights of 1.0, 1.5, and 2.0 m place the IK target 0.29–1.29 m above the natural EE position when the arm is mounted on the ground plane.

This exceeds the arm's reach. The resolution is to mount the arm on a pedestal: setting `base_position` $= (0, 0, z_{\text{release}} - 0.5)$ places the neutral EE height at $z_{\text{release}} + 0.21\,\text{m}$, so the IK target at z_{release} lies approximately 0.21 m below the neutral configuration—well within the arm's kinematic range. Table 6 summarises the per-height parameter choices.

Table 6: Per-configuration parameters for Study 3 (PBE-B/C/D). All three share $u_M = 3.5\,\text{m/s}$, $N_{\text{exp}} = 5$, $\ell_c = 0.5\,\text{m}$, $\ell_s^{\text{init}} = [1.0, 1.0]$, stratified exploration over $[0, u_M]$.

Config	z_{rel}	`base_position`	ℓ_M	T
PBE-B	1.0 m	$(0, 0, 0.5)$	1.90 m	0.85 s
PBE-C	1.5 m	$(0, 0, 1.0)$	2.15 m	0.95 s
PBE-D	2.0 m	$(0, 0, 1.5)$	2.35 m	1.00 s

7.3 Implementation

`mc-pilot-pb-elevated/` was created by copying `mc-pilot-pybullet/` (which contains all PyBullet plumbing) and adapting it to the elevated hyperparameter sets. Noise-specific scripts (`test_mc_pilot_pb_B.py`, `test_mc_pilot_pb_C.py`) were removed; the zero-noise base (`arm_noise=None`) is used throughout, matching the conditions of Study 1 exactly. Two new constructor kwargs propagate the pedestal and velocity-limit settings:

- `PyBulletThrowingSystem(base_position=..., vel_limit_multiplier=1.5)`
- `ArmController(..., vel_limit_multiplier=1.5)` via `self._qd_max = _IIWA_QD_MAX × vel_limit_multiplier`

These changes preserve the default behaviour of `mc-pilot-pybullet/` (multiplier = 1.0, base at origin) and require no modifications to the RL algorithm or GP code.

Verification. Running `standalone_throw.py -direct -speed 3.5 -z_release 1.0` confirms $v_{\text{release}} = 3.500\,\text{m/s}$ and a ball landing at 2.099 m. This compares to the NumPy elevated Study 1 verified maximum range of $\approx$ 1.97 m for Config B—a 5–10% upward offset attributable to per-step drag application differences between PyBullet `applyExternalForce` and the NumPy integrator. Because the GP is fitted to actual PyBullet throws (not NumPy predictions), it learns the correct speed-to-range mapping for the PyBullet physics. The margin $\ell_M = 1.90\,\text{m} < 2.10\,\text{m}$ (PyBullet max range) provides 0.2 m of coverage room, ensuring all sampled targets remain reachable.

7.4 PyBullet–NumPy Physics Discrepancy

Drag applied via `applyExternalForce` per simulation step accumulates small numerical differences compared with the closed-form NumPy midpoint rule. At $u_M = 3.5\,\text{m/s}$ and $z_{\text{rel}} = 1.0\,\text{m}$ the discrepancy is $\approx 6\%$ in landing range. This does not impair learning: the GP observes PyBullet ground truth during both exploration and policy trials, so both the dynamics model and the policy converge to the PyBullet physics, not the NumPy approximation. The effect on cross-study comparisons is that trained policies from `mc-pilot-elevated/` and `mc-pilot-pb-elevated/` will produce slightly different release speeds for the same target, but will achieve comparable hit rates.

7.5 Visualisation Pipeline

After training completes, `demo_pybullet_gui.py` loads the saved policy from `log.pkl` (reading `parameters_trial_list[-1]`, which stores {log_lengthscales, centers, f_linear.weight}) and replays N throws in PyBullet GUI mode. Each throw renders the full arm trajectory (IK + cubic spline), ball flight with drag, and landing annotations:

- `addUserDebugLine` trajectory traces (ball path in 3D)
- Red sphere at the target location
- Green (hit) or orange (miss) sphere at the landing location, with error distance printed as GUI text

Optional flags: `-record output.mp4` (via `startStateLogging(STATE_LOGGING_VIDEO_MP4)`) and `-slow N` to multiply `time.sleep(dt)` for slower playback. Command: `python demo_pybullet_gui.py -log_path results_mc_pilot_pbe_B/1`.

7.6 Expected Results

[DATA NEEDED: Run Study 3 training scripts for PBE-B/C/D (seed=1, 10 trials each) and record: per-trial hit rate, per-throw landing error, convergence cost trajectory.]

Based on the design, PBE-B/C/D are expected to replicate the 10/10 hit rates of Study 1 Configs

B/C/D-Strat: the hyperparameters are identical, the physics equations are the same (Eq. 35 drag, explicit velocity setting), and the GP learns from PyBullet ground truth rather than an external model. The 5–10% PyBullet drag offset shifts the optimal release speed slightly but does not affect policy convergence quality.

8 Study 4: Wind Robustness

[WIP: Study 4 is planned but not yet executed. This section describes the intended experimental design; results will be added in the final version.]

Outdoor or atrium-deployed throwing systems must operate under non-zero ambient airflow. Study 4 extends the drag model (Eq. 4) with a wind force term

$$\mathbf{F}_W = \tfrac{1}{2}\rho\, C_D\, A\, \|\mathbf{w}\|\, \mathbf{w}, \tag{19}$$

where $\mathbf{w} \in \mathbb{R}^3$ is the ambient wind velocity vector. Three regimes are planned:

1. **Constant wind.** A fixed $\mathbf{w} = (w_x, 0, 0)$ throughout all throws. The GP should absorb the systematic bias after a few trials, since constant wind shifts every landing by the same horizontal offset.

2. **Gaussian gusts.** $\mathbf{w}$ is drawn independently at the start of each throw from $\mathcal{N}(\boldsymbol{\mu}_w, \Sigma_w)$. This is analogous to the velocity-slip noise studied in Study 2: the policy must compensate the mean wind while the variance sets an irreducible landing-error floor.

3. **Per-timestep turbulence.** $\mathbf{w}$ is resampled at each integration step, approximating high-frequency turbulent fluctuations. The key question is whether the GP can model the resulting non-Gaussian landing distribution or whether wind state must be explicitly included in the policy input.

The primary scientific question is whether augmenting the policy input with an observed wind state $\mathbf{w}$ (*e.g.*, from an anemometer) improves hit rate compared with a wind-blind policy that relies on GP adaptation alone. A secondary question is whether the zero-mean unlearnability result (Section 9.2) extends to zero-mean gust distributions, or whether the momentum coupling through Eq. 19 makes even symmetric gusts partially learnable.

9 Discussion

9.1 Geometry-Dependence of Algorithm Defaults

The published Table 1 hyperparameters of MC-PILOT Turcato et al. [2025] are calibrated to a Franka Panda arm on a flat floor with a fixed release height and a target range of approximately 1.0 m. Our study quantifies how several of these defaults degrade silently when the geometry changes.

RBF lengthscale. The default $\ell_s = 1.0\,\mathrm{m}$ is appropriate when the target arc spans $\sim$1 m in Euclidean distance. At Config E ($z_{\mathrm{rel}} = 0.0\,\mathrm{m}$, target range 0.35 m), the same lengthscale yields RBF sensitivity $\exp\!\left(-\frac{1}{2}(0.35/1.0)^2\right) = 0.941$ between the two extremes of the target domain—the policy produces an effectively constant output regardless of target position. Four successive failed experiments (Section 5) were required to isolate this as the root cause; once identified, replacing $\ell_s = 1.0$ with $\ell_s = 0.15\,\mathrm{m}$ (maintaining sensitivity ≈ 0.066, comparable to the working configs) immediately recovered 10/10 hits. The general rule is $\ell_s \approx 0.15 \times \mathrm{target_range}$ (Appendix C).

Cost lengthscale. The paper's $\ell_c = 0.1\,\mathrm{m}$ generates useful gradients only when the policy already delivers within $\sim$0.15 m of the target. With random exploration, early landing errors span 0.3–1.5 m, saturating the cost at $c \approx 1.0$ and driving gradients to zero. Widening to $\ell_c = 0.5\,\mathrm{m}$ keeps gradients alive across the full exploration error range; then the cost can be tightened only after the GP becomes accurate enough that near-misses are below 0.2 m. Attempting to revert to $\ell_c = 0.1\,\mathrm{m}$ after 5 trials still yields 0/10 hits because early-trial errors remain too large.

Exploration coverage. At $z_{\mathrm{rel}} = 1.0\,\mathrm{m}$, random exploration draws five speeds from $[0, u_M]$; an unlucky seed (seed=1, Config B) draws four low speeds ($v < 1.5\,\mathrm{m/s}$), leaving the GP uninformed about the high-speed dynamics needed for far targets. The policy converges to a max-speed local minimum (cost plateau at 0.70–0.73, 0/10 hits). The identical seed succeeds with stratified exploration, which partitions $[0, u_M]$ into N_{exp} equal bands and samples one throw per band, guaranteeing full coverage in N_{exp} throws.

9.2 Zero-Mean Noise as GP Regularisation

An unexpected finding emerged from the noise study: the additive observation noise $\sigma = 0.04\,\mathrm{m/s}$ (introduced to model velocity scatter) is not merely a noise source for the aware/naive comparison—it is a necessary regulariser for GP hyperparameter estimation.

When $\sigma = 0$ and exploration produces five noiseless, deterministic ballistic trajectories, maximum likelihood estimation (MLE) of GP hyperparameters finds that the mathematically easiest fit is a nearly flat function with effectively infinite lengthscales (observed values: $\ell \approx 1791\,\mathrm{m}$, $1602\,\mathrm{m}$, $357\,\mathrm{m}$ over a domain of $\sim 1\,\mathrm{m}$ in position and $\sim 3\,\mathrm{m/s}$ in velocity). A GP with lengthscale $\gg$ domain width predicts the same landing position regardless of launch speed, making policy gradients effectively zero. With $\sigma = 0.04$, MLE must find finite lengthscales that explain why throws with similar speeds land at different positions; the resulting structure is what allows the aware policy to learn the $1/(1-\alpha)$ compensation multiplier (Proposition 2 and Appendix B).

Proposition 2 (Zero-mean unlearnability)**.** *Let $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \Sigma_\epsilon)$ be additive zero-mean noise on the release velocity. The optimal policy π^*_{aware} minimising the noise-averaged cost $J(\theta) = \mathbb{E}_{\boldsymbol{\epsilon}}\big[c(\mathbf{p}_T)\big]$ is identical to the noiseless optimal policy π^*_{naive}. Consequently, noise awareness provides no improvement in aim under zero-mean noise.*

The proof is given in Appendix B.

The practical implication for simulation-only experiments is that practitioners should add a small artificial observation noise $\sigma_n > 0$ to GP training data even in noiseless simulators, treating it as GP regularisation rather than a physical effect. A value of $\sigma_n \approx 0.04\,\mathrm{m/s}$ is sufficient for the throwing domain.

9.3 Decoupled Arm–RL Pipeline

Setting the ball velocity explicitly via `resetBaseVelocity` at release decouples arm tracking quality from GP training quality. Even with substantial joint-tracking error (the iiwa7 position controller achieves $\approx 7\%$ EE velocity tracking error in our experiments), the ball always receives exactly $\mathbf{v}_{\text{cmd}}$, and GP training data are never corrupted by kinematic imprecision.

This design is not merely a simulation convenience. It corresponds physically to a gripper that can be programmed to release at a precisely measured EE velocity (read from the joint encoder forward kinematics). In that case, the delivered velocity equals the measured velocity at the release instant, and the arm controller only needs to bring the EE to the desired pose and velocity—not to track a continuous trajectory with sub-millimetre precision. The RL algorithm's data efficiency guarantees are preserved regardless of trajectory-tracking quality, provided the release velocity is accurately measured or commanded.

9.4 Limitations

Simulation only. All experiments are conducted in simulation. Hardware validation on a physical manipulator is needed to confirm that the learned policies transfer and that the 5–10% PyBullet–NumPy drag discrepancy does not compound with real-world effects.

Fixed launch angle. The elevation angle $\alpha = 35°$ is fixed across all experiments. Joint optimisation of α and release speed would increase the reachable target volume and potentially improve data efficiency by concentrating GP data along the optimal ballistic arc.

Single-seed statistics. Most results are reported for seed=1 only; Study 1 Config B was also evaluated at seed=2 for robustness verification. Conference-level statistical claims require ≥ 5 seeds to bound performance variance. The seed=1 vs seed=2 comparison for Config B (random exploration) illustrates that hit rate can vary between 0/10 and 10/10 for an unlucky vs lucky seed, confirming that single-seed results should be read as existence proofs rather than expected-case performance.

Particle freeze approximation. Once a particle's z-coordinate falls below zero, its state is frozen at the previous timestep rather than interpolated to the exact $z = 0$ crossing. This introduces a systematic over-estimation of the landing distance proportional to the vertical velocity at the last step above ground. For the simulation timestep $T_s = 0.02\,\mathrm{s}$ and typical vertical velocities of 1–3 m/s at landing, the frozen-step error is 2–6 cm per particle. Landing detection in the actual rollout uses linear interpolation to correct this, so the training data are accurate; the approximation affects only the Monte Carlo cost estimate used for gradient computation (Appendix D).

10 Conclusion

We presented a diagnostic reproduction of MC-PILOT Turcato et al. [2025] in a self-contained NumPy and PyBullet simulator, and conducted three focused studies to characterise how the algorithm behaves outside its original calibration geometry.

Three technical contributions emerge from this work. First, we diagnosed and corrected four non-obvious failure modes in the baseline implementation: a broken gradient path from cost to policy parameters (T_control unit mismatch), cost saturation from excessive lengthscale ℓ_c relative to exploration error magnitude, a wasted simulation horizon due to particles propagating after landing, and an index-range error in the early stopping infrastructure. Each failure was identified by instrumentation, understood through the algorithm structure, and resolved with a minimal targeted fix.

Second, two transferable design rules emerged from Study 1. The *lengthscale-to-range scaling rule*—$\ell_s \approx 0.15 \times \text{target_range}$—prevents the RBF policy from becoming blind over narrow target domains, a failure mode that is invisible from the cost curve alone. The *stratified exploration rule* partitions the speed range into equal bands and samples one throw per band, providing seed-robust GP coverage in the same number of exploration throws as random sampling while eliminating the max-speed local minima that trapped random exploration on elevated configs.

Third, the release-height generalisation study demonstrates 100% hit rates on $z \in \{0.5, 1.0, 1.5, 2.0\}\,\text{m}$ with mean landing errors below 35 mm across all four configurations, and 10/10 hits on Config E ($z = 0.0\,\text{m}$) after iterative diagnosis through five progressive fixes. These results confirm that the corrected algorithm generalises across a 2 m range of release heights without any per-height tuning beyond the three parameters (ℓ_M, T, ℓ_s) that are geometrically motivated.

Wind robustness is the natural next step: extending the drag model with a wind force term (Eq. 19) and studying whether the GP can absorb systematic wind bias within the same five-throw exploration budget used for the static case. Open questions include few-shot policy transfer across geometries (can a policy trained at $z = 1.0\,\text{m}$ be warm-started for $z = 2.0\,\text{m}$ with fewer trials than training from scratch?) and wind-state augmentation of the policy input, which would transform the single-shot speed map into a full contextual policy conditioned on measured airflow.

References

Mitsubishi Electric Research Laboratories. MC-PILCO: Monte carlo policy improvement with learning-based control. `https://github.com/merlresearch/MC-PILCO`, 2022. GitHub repository.

N. Turcato, G. Giacomuzzo, R. Carli, and D. Romeres. Data efficient robotic object throwing with model-based reinforcement learning. *arXiv preprint arXiv:2502.05595*, 2025. URL `https://arxiv.org/abs/2502.05595`.

A Full Hyperparameter Table

Table 7 consolidates every configuration evaluated across all three studies. Parameters not listed for a config are identical to the baseline (Config A).

Table 7: Complete hyperparameter table for all experimental configurations. Shared across all: M=400, N_b=250, u_M=3.5 m/s, ℓ_m=0.75 m, γ_M=$\pi/6$, T_s=0.02 s, N_{opt}=1500, α_{launch}=35°, ball (m=0.0577 kg, r=0.0327 m). "Strat" = stratified exploration; "Rnd" = random.

Config	Sub-project	z_{rel}	ℓ_M	T	ℓ_c	ℓ_s^{init}	N_{exp}	Exploration	Result
A	mc-pilot/	0.5 m	1.75 m	0.70 s	0.5 m	[1.0, 1.0]	5	Rnd	5/5 hits
B	mc-pilot-elevated/	1.0 m	1.90 m	0.85 s	0.5 m	[1.0, 1.0]	5	Rnd	0/10 (seed=1); 10/10 (
B-Strat	mc-pilot-elevated/	1.0 m	1.90 m	0.85 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	10/10
C-Strat	mc-pilot-elevated/	1.5 m	2.15 m	0.95 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	10/10
D-Strat	mc-pilot-elevated/	2.0 m	2.35 m	1.00 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	10/10
E-Strat	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	0/10
E-Strat2	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.5 m	[1.0, 1.0]	5	Strat $[2.5, 3.5]$	0/10
E-Strat3	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.5 m	[1.0, 1.0]	10	Strat $[2.5, 3.5]$	0/10
E-Strat4	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.25 m	[1.0, 1.0]	10	Strat $[2.5, 3.5]$	0/10
E-Strat5	mc-pilot-elevated/	0.0 m	1.10 m	0.55 s	0.25 m	[0.15, 0.15]	10	Strat $[2.5, 3.5]$	10/10
PBE-B	mc-pilot-pb-elevated/	1.0 m	1.90 m	0.85 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	**[DATA NEEDED: r**
PBE-C	mc-pilot-pb-elevated/	1.5 m	2.15 m	0.95 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	**[DATA NEEDED: r**
PBE-D	mc-pilot-pb-elevated/	2.0 m	2.35 m	1.00 s	0.5 m	[1.0, 1.0]	5	Strat $[0, u_M]$	**[DATA NEEDED: r**

B Zero-Mean Unlearnability: Proof Sketch

Proposition 3 (Zero-mean unlearnability, restated)**.** *Let $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \Sigma_\epsilon)$ be additive noise on the commanded release velocity. The optimal policy under noise-awareness is identical to the noiseless optimal policy.*

Proof sketch. Let $f : \mathbb{R}^3 \to \mathbb{R}^2$ denote the deterministic landing map (position as a function of release velocity), assumed smooth and invertible near the optimum. By first-order Taylor expansion around $\mathbf{v}^*$:

$$f(\mathbf{v}^* + \boldsymbol{\epsilon}) \approx f(\mathbf{v}^*) + J_f \boldsymbol{\epsilon}, \qquad J_f = \tfrac{\partial f}{\partial \mathbf{v}}\big|_{\mathbf{v}^*}.$$

The expected landing position is

$$\mathbb{E}_\epsilon[f(\mathbf{v}^* + \boldsymbol{\epsilon})] = f(\mathbf{v}^*) + J_f \mathbb{E}[\boldsymbol{\epsilon}] = f(\mathbf{v}^*),$$

since $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$. The noise-averaged cost gradient with respect to the policy output $\mathbf{v}^*$ is

$$\nabla_{\mathbf{v}^*} J_{\text{aware}} = \nabla_{\mathbf{v}^*} \mathbb{E}_\epsilon[c(f(\mathbf{v}^* + \boldsymbol{\epsilon}))] = J_f^\top \nabla_{\mathbf{p}} c\big(f(\mathbf{v}^*)\big),$$

which is identical to $\nabla_{\mathbf{v}^*} J_{\text{naive}}$ computed with $\boldsymbol{\epsilon} = \mathbf{0}$. Therefore, the gradient is zero at the same $\mathbf{v}^*$ for both the aware and naive objectives, and the optimal policies coincide. The noise increases the variance of c (and thus the honest cost value) but does not shift the optimal aim. □

Remark 1. *The result relies on linearity of f near the optimum and zero mean of $\boldsymbol{\epsilon}$. For skewed or heavy-tailed noise distributions the approximation breaks and a genuine gap between aware and naive policies can emerge.*

C Lengthscale Scaling Rule Derivation

The RBF policy (Eq. 2) uses a squared-exponential kernel with lengthscale ℓ_s. The sensitivity between two target positions $\mathbf{P}_1$ and $\mathbf{P}_2$ at Euclidean distance d is

$$S(d,\, \ell_s) = \exp\Big(-\tfrac{d^2}{2\ell_s^2}\Big).$$

Config A (target range $\approx 1.0\,\mathrm{m}$, $\ell_s = 1.0\,\mathrm{m}$) achieves sensitivity $S_A = \exp(-0.5) \approx 0.607$ between the two extremes of its domain.

For a new configuration with target range R, we require the same sensitivity:

$$S(R,\, \ell_s^{\mathrm{new}}) \stackrel{!}{=} S_A = 0.607.$$

Setting $\exp(-R^2/(2\ell_s^2)) = \exp(-0.5)$ gives

$$\frac{R^2}{2\ell_s^2} = \frac{1}{2} \implies \ell_s = R.$$

Hence $\ell_s = R$ for all target ranges—but in practice, the paper's fixed $\ell_s = 1.0\,\mathrm{m}$ is used even when $R \ll 1.0\,\mathrm{m}$. The issue is that Config A has $R \approx 1.0\,\mathrm{m}$, so $\ell_s = R$ happens to hold by coincidence. For Config E with $R = 0.35\,\mathrm{m}$, setting $\ell_s = 0.35\,\mathrm{m}$ recovers $S = 0.607$. In our experiments, $\ell_s = 0.15\,\mathrm{m}$ (which gives $S = 0.066$, lower than 0.607) was found to work well because even sharper sensitivity promotes faster learning when the GP has limited exploration data. The conservative recommendation is $\ell_s = R$; the empirical finding is that $\ell_s \approx 0.15R$ is safe and often faster.

D Particle Freeze Approximation at Landing

The GP propagates M particles forward in time. Once a particle's z-coordinate satisfies $z \leq 0$, its state is frozen at the last step above ground and the cost is evaluated at that frozen landing position for all remaining timesteps.

This introduces a systematic over-estimation of the horizontal landing distance proportional to the residual horizontal velocity at the frozen step. For timestep $T_s = 0.02\,\mathrm{s}$ and a typical vertical velocity of $2\,\mathrm{m/s}$ at impact, the true ground crossing occurs $\Delta t = z_{\mathrm{last}}/v_z \approx 0.01\,\mathrm{s}$ before the frozen step, during which the ball travels an additional $\Delta x \approx v_x \cdot \Delta t \approx 2$–$4\,\mathrm{cm}$ horizontally (for $v_x \approx 2$–$4\,\mathrm{m/s}$).

In actual rollouts (NumPy and PyBullet), landing is detected at $z \leq 0$ and refined by linear interpolation to the exact $z = 0$ crossing, so training data are not affected by this error. The approximation applies only to the Monte Carlo particles used for the gradient estimate inside `apply_policy`, where implementing per-particle interpolation would break batched tensor operations. The resulting cost estimates are slightly pessimistic (the policy is penalised for landing a few centimetres beyond the target when the true landing is closer), but the bias is consistent across particles and policy updates, so optimisation still converges to the correct policy. Reducing T_s or implementing a soft landing indicator (a smooth sigmoid of z rather than a hard step) would eliminate this error at the cost of increased computation.